

Euro School - Guide d'installation Windows et de deployment

1) Presentation de la solution

Euro School est une application web full-stack basee sur :

- **Backend** : Laravel 12 (PHP 8.2+)
- **Frontend** : React 18 + Vite 7 + Tailwind CSS 4
- **Base de donnees** : SQLite (par default) ou MySQL/MariaDB
- **Authentification** : Laravel Sanctum (API avec token)
- **Bibliotheques UI** : Chart.js, jsPDF, html2canvas

2) Communication entre les composants

2.1 Architecture d'execution

- Le navigateur charge la SPA via Laravel (`resources/views/app.blade.php`).
- React est initialise depuis `resources/js/app.jsx`.
- Le client API frontend (`resources/js/api.js`) appelle les endpoints `/api`.
- Les routes Laravel (`routes/api.php`) dirigent vers les controleurs.
- Les controleurs utilisent Eloquent pour lire/crire dans la base.

2.2 Flux technique

1. Action utilisateur dans l'interface React.
2. Requete Axios vers `/api/...` avec token Bearer.
3. Middleware Laravel (auth + role) valide l'accès.
4. Le controleur traite la requete et renvoie du JSON.
5. React met a jour l'etat et rafraichit l'affichage.

3) Plateforme technique requise (Windows)

Prerequis minimum

- **OS** : Windows 10/11 (64 bits)
- **PHP** : 8.2+ avec extensions Laravel usuelles :

- `openssl`, `pdo`, `pdo\_sqlite` (ou `pdo\_mysql`), `mbstring`, `tokenizer`, `xml`, `ctype`, `json`, `fileinfo`
- **Composer** : 2.x
- **Node.js** : 20+ (LTS recommandée)
- **npm** : 10+
- **Git** : version récente

Outils recommandés

- Terminal : PowerShell
- Editeur : Cursor / VS Code
- Navigateur : Chrome ou Edge

4) Installation locale sous Windows (développement)

Étape 1 - Cloner le dépôt

```

```powershell
git clone https://github.com/ericmeduse-dev/euro-school.git
cd euro-school
```

```

Étape 2 - Installer les dépendances

```

```powershell
composer install
npm install
```

```

Étape 3 - Initialiser l'environnement

```

```powershell
copy .env.example .env
php artisan key:generate
```

```

Étape 4 - Configurer la base

Le `.env.example` utilise SQLite par default.

Creer le fichier SQLite s'il n'existe pas :

```
```powershell
if (!(Test-Path database\database.sqlite)) { New-Item database\database.sqlite -ItemType File }
```
```

Executer migrations + seed :

```
```powershell
php artisan migrate
php artisan db:seed
```
```

Etape 5 - Demarrer l'application

Terminal A :

```
```powershell
php artisan serve
```
```

Terminal B :

```
```powershell
npm run dev
```
```

Acces :

- `http://127.0.0.1:8000`

> Important : si Vite est arrete mais que `public/hot` existe, vous pouvez avoir un ecran blanc.
Supprimez `public/hot` puis lancez `npm run build`.

5) Execution locale type production (sans Vite dev server)

```
```powershell
npm run build
php artisan serve
```
```

Les assets compiles sont servis depuis `public/build`.

6) Deploiement sur 3 plateformes differentes

Plateforme A - VPS Linux (Ubuntu + Nginx + PHP-FPM + MySQL)

Adapte pour

- Controle total, bonnes performances, cout maitrise.

Etapas principales

1. Installer Nginx, PHP 8.2, Composer, Node.js, MySQL.
2. Cloner le projet dans `/var/www/euro-school`.
3. Executer :
 - `composer install --no-dev --optimize-autoloader`
 - `npm ci && npm run build`
 - `php artisan key:generate`
 - `php artisan migrate --force`
4. Configurer les droits sur `storage` et `bootstrap/cache`.
5. Configurer Nginx avec racine web sur `public/`.
6. Ajouter un superviseur de process pour les queues si necessaire.

Bonnes pratiques

- `APP\_ENV=production`, `APP\_DEBUG=false`
- SSL (Let's Encrypt) + sauvegardes regulieres

Plateforme B - Hebergement mutualise (cPanel/Apache)

Adapte pour

- Hebergement simple avec maintenance reduite.

Etapas principales

1. Uploader les fichiers du projet.
2. Pointer le domaine/sous-domaine vers le dossier `public/`.

3. Configurer ``.env`` avec les identifiants base hebergeur.
4. Lancer les migrations (``php artisan migrate --force``) si SSH disponible.
5. Si Node n'est pas disponible sur l'hebergeur :
 - compiler en local (``npm run build``)
 - uploader ``public/build``.

Bonnes pratiques

- Verifier les droits d'ecriture (``storage``, ``bootstrap/cache``).
- Verifier version PHP 8.2+ et regles de rewrite Apache.

Plateforme C - Docker (Windows/WSL2 ou serveur Linux)

Adapte pour

- Environnements reproductibles et travail d'equipe.

Architecture type

- ``app`` (PHP-FPM + Laravel)
- ``web`` (Nginx)
- ``db`` (MySQL/MariaDB) ou volume SQLite
- etape optionnelle de build frontend

Etapas principales

1. Construire et lancer les conteneurs :
 - ``docker compose up -d --build``
2. Installer les dependances backend si necessaire :
 - ``composer install``
3. Lancer les migrations :
 - ``php artisan migrate --force``
4. Compiler les assets frontend :
 - ``npm ci && npm run build``

Bonnes pratiques

- Volumes montes en dev, images immuables en production.
- Secrets hors du depot Git.

7) Variables d'environnement (production)

- `APP\_ENV=production`
- `APP\_DEBUG=false`
- `APP\_URL=https://votre-domaine`
- `DB\_CONNECTION`, `DB\_HOST`, `DB\_PORT`, `DB\_DATABASE`, `DB\_USERNAME`, `DB\_PASSWORD`
- `SESSION\_DRIVER`, `CACHE\_STORE`, `QUEUE\_CONNECTION`

Optimisation :

```
```powershell
php artisan config:cache
php artisan route:cache
php artisan view:cache
```
```

8) Depannage

Ecran blanc

- En mode dev : verifier `npm run dev`.
- Hors mode dev : supprimer `public/hot` puis lancer `npm run build`.

Erreurs API 500

- Verifier `storage/logs/laravel.log`.
- Verifier que les migrations ont bien ete executees.

Erreurs de permissions

- Verifier droits d'ecriture sur `storage/` et `bootstrap/cache/`.

9) Checklist de mise en production

- [] Variables d'environnement production configurees
- [] SSL actif
- [] Base migree (et seedee si necessaire)

- ☐ Assets frontend compiles
- ☐ Caches Laravel optimises
- ☐ Sauvegardes et monitoring en place
- ☐ Logs d'erreurs verifiés